UNIVERSITÉ CÔTE D'AZUR

Microservices with Spring Boot

Basics

MBDS

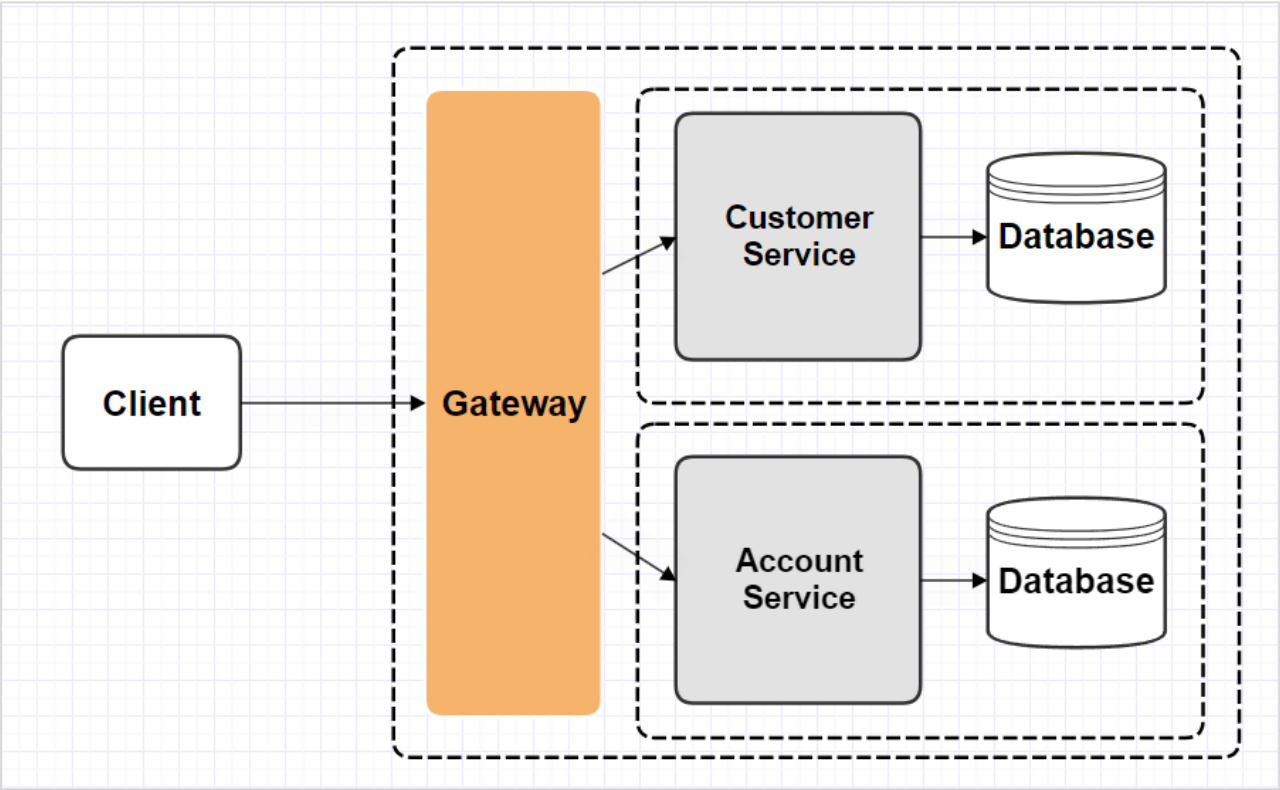
www.mbds-fr.org

Microservices

Project architecture, edge services & dependencies

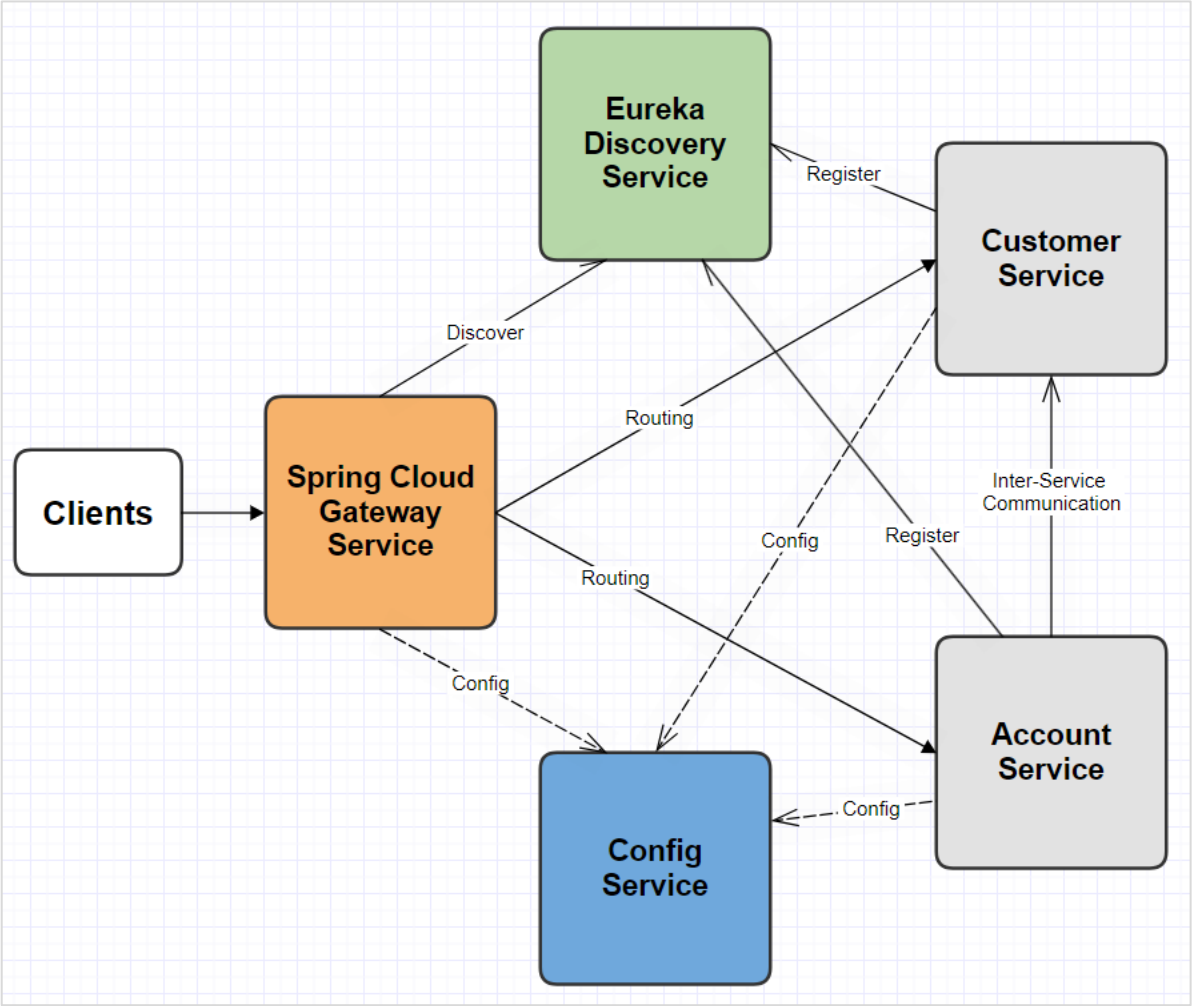
Microservice – Project

- Simple project to demonstrate Microservice usage
- Banking Service
  - Customer microservice
  - Account microservice



Microservice – Edge Microservices

- Gateway
- Discovery
- Config



Microservice – Dependencies

- Classic
  - Spring Data JPA
  - H2 database
  - Lombok
  - OpenAPI (Swagger)
- Spring Cloud
  - Gateway (routing, monitoring, metrics, resilience, security)
  - Discovery (Netflix Eureka Client / Server)
  - Config (Client / Server)
- Actuator (monitoring)
- OpenFeign (declarative REST client)
- Resilience4J (circuit breaker)

Microservices

New project

New project

- Create empty project
- Add a new module for each service
  - Customer
  - Account
- For each module
  - Java 17+
  - Gradle / Maven
  - Dependencies

Customer Service

- Create new module
- Name : customer
- Dependencies
  - Spring web
  - JPA
  - H2
  - Lombok
  - Rest Repositories

Customer Service – Project Structure

- Packages
  - Entities
  - Controllers
  - Repositories
  - Services
  - Web
  - Dtos
  - Mappers
  - Models
  - …

Customer Service – Entity

- Basic attributes
- Don’t forget
  - @Entity Annotation
  - Id definition
  - Lombok annotations

Customer Service – Entity

```java
// Annotations Lombok
@Getter @Setter @ToString @NoArgsConstructor @AllArgsConstructor @Builder
// @Data - Incompatible avec JPA - due to @EqualsAndHashCode implementation
@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
}
```

Customer Service – Repository

- @Repository annotation (optional)
- Use JpaRepository
- @RepositoryRestResource
  - Declarative API creation
  - Don’t need to create a controller
  - Use only for POC & prototypes

Customer Service – Repository

```
// Spring Data REST > fait de l'entité une ressource REST
// Non utilisé pour faire un peu plus propre > On doit créer un controller
//@RepositoryRestResource
@Repository
public interface CustomerRepository extends JpaRepository<Customer, Long> {

}
```

Customer Service – Controller

- Dependency injection for the repository
  - Ideally on constructor
- Methods
  - getCustomers
  - getCustomerById

Customer Service – Controller

```java
@RestController
public class CustomerController {
    private final CustomerRepository customerRepository;

    public CustomerController(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    @GetMapping("/customers")
    public List<Customer> getCustomers() {
        return customerRepository.findAll();
    }

    @GetMapping("/customer/{id}")
    public Customer getCustomer(@PathVariable Long id) {
        return customerRepository.findById(id).orElse(null);
    }
}
```


www.mbds-fr.org

Customer Service – Config

- Application name
- Server port
- Data source URL
- Activate h2 console

```yaml
#src/resources/application.yml
spring:
  application:
    name: customer
  datasource:
    url: jdbc:h2:mem:customer
  h2:
    console:
      enabled: true
server:
  port: 8080
```

Customer Service – Testing

- CommandLineRunner in the main class
  - Functional interface (one run method only)
- @Bean annotation to make sure it’s ran at startup
- Customer repository injection
- Create new instances

Customer Service – Testing

```
@Bean
CommandLineRunner commandLineRunner(CustomerRepository customerRepository) {
    return args -> {
        List<Customer> customersList = List.of(
                Customer.builder()
                        .firstName("Elodie").lastName("Bantos")
                        .email("elodie.bantos@etu.univ-cotedazur.fr")
                        .build(),
                Customer.builder()
                        .firstName("Yue").lastName("Guo")
                        .email("yue.guo@etu.univ-cotedazur.fr")
                        .build(),
                Customer.builder()
                        .firstName("Valeriia").lastName("Lapshina")
                        .email("valeriia.lapshina@etu.univ-cotedazur.fr")
                        .build(),
                Customer.builder()
                        .firstName("Dounia").lastName("Zoubid")
                        .email("dounia.zoubid@etu.univ-cotedazur.fr")
                        .build()
                );
        customerRepository.saveAll(customersList);
    };
}
```

Customer Service – Testing

- End points
  - Customers
  - Customer by Id
- h2
- Add Open API dependencies
  - Check /swagger-ui/index.html

Account Service

New Module

Account Service

- Redo with different Entity

```
@Getter @Setter @ToString @NoArgsConstructor @AllArgsConstructor @Builder
@Entity
public class Account {
    @Id
    private String id;
    private Double balance;
    private LocalDate dateCreated;
    @Enumerated(EnumType.STRING)
    private CurrencyType currency;
    @Transient
    private Customer customer;
    private Long customerId;
}
```


www.mbds-fr.org

Account Service

- Define Enums in the appropriate package

```
public enum CurrencyType {
    EUR, USD
}
```

Account Service

- Can use a model to handle the external “Customer” resource
- Generate useful methods with Lombok

```
@Getter @Setter @ToString @NoArgsConstructor @AllArgsConstructor
public class Customer {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
}
```

Account Service

- Then
  - Repository
  - Controller
  - Config
  - Create default data
  - Test

Account Service - Repository

- Classic repository but mind about the primary key type : String

```
public interface AccountRepository extends JpaRepository<Account, String> {
}
```


www.mbds-fr.org

Account Service - Controller

UNIVERSITÉ CÔTE D'AZUR

- Classic repository but mind about the primary key type : String

```
@RestController
public class AccountController {

    private AccountRepository accountRepository;

    public AccountController(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;

    @GetMapping("/accounts")
    public List<Account> getAllAccounts()
    {
        return accountRepository.findAll();
    }

    @GetMapping("/account/{id}")
    public Account getAccountById(@PathVariable String id)
    {
        return accountRepository.findById(id).orElse(null);
    }
}
```

Account Service - Config

- Classic config file
  - Port
  - Enable console
  - Datasource url
  - Application name
- Can use either
  - application.properties
  - application.yml

```
server.port=8081
spring.h2.console.enabled=true
spring.datasource.url=jdbc:h2:mem:devDb
spring.application.name=account
```


www.mbds-fr.org

Account Service – Default Data

- Same story as Customer Service

```
@Bean
CommandLineRunner commandLineRunner(AccountRepository accountRepository) {
    return args -> {
        List<Account> accountList = List.of(
                    Account.builder()
                            .id(UUID.randomUUID().toString())
                            .balance(100D)
                            .currencyType(CurrencyType.EUR)
                            .dateCreated(LocalDate.now())
                            .customerId(1L)
                            .build(),
                    Account.builder()
                            .id(UUID.randomUUID().toString())
                            .balance(200D)
                            .currencyType(CurrencyType.EUR)
                            .dateCreated(LocalDate.now())
                            .customerId(2L)
                            .build()
        );
        accountRepository.saveAll(accountList);
    };
}
```

To be continued

Gateway, Discovery, Config, Circuit breaker, Docker …

Account Service – Testing

- End points
  - Accounts
  - Account by Id
- h2
- Add Open API dependency
  - Check /swagger-ui/index.html

Account Service Improvements

Link with Customer Service

Account Service Improvements

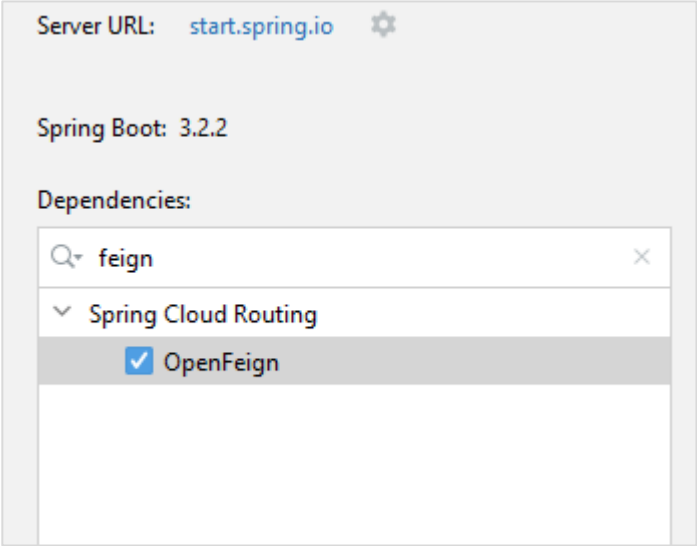
UNIVERSITÉ CÔTE D'AZUR

- Customer id is set manually
- Need to work with real Customer ids
- Also, Customer model from Account Service POV is not set
  - Need to fix this
- To enable the link, we will use Feign : a declarative REST client
- First need to add the dependency to Account Service
  - Search & add Feign Client in the dependencies declared in the build.gradle file

Account Service Improvements

```
dependencies { Edit Starters...
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.cloud:spring-cloud-starter-openfeign'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'com.h2database:h2'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

Account Service Improvements




```
dependencies {
    …
    implementation 'org.springframework.cloud:spring-cloud-starter-openfeign'
    …
}
```


www.mbds-fr.org

Account Service Improvements

- Then add new annotation to the main “AccountApplication.java”
- This annotation will tell Spring to extends the component discovery mechanism to the Feign annotated file we will create
  - Without this annotation Feign annotations won’t be available for the dependency injection

```
@SpringBootApplication
@EnableFeignClients
public class AccountApplication {
```

Account Service Improvements

- Now we need to create a new Feign Client
  - Will be an interface declaring methods corresponding to Customer Controller methods
  - Also need to set the name of the targeted service as well as the url to reach it …

```
@FeignClient(name = "customer", url = "http://localhost:8080/")
public interface CustomerClient {

    @GetMapping("/customer/{id}")
    public Customer getCustomer(@PathVariable Long id);

    @GetMapping("/customers")
    public List<Customer> getCustomers();
}
```


www.mbds-fr.org

Account Service Improvements

UNIVERSITÉ CÔTE D'AZUR

- Now we can use this interface / Feign client / Proxy to request the Customer Service and build proper accounts
- Don't forget to inject the newly created proxy in the controller
- Use constructor dependency injection to make it available
  - If the injection returns an error, you forgot to add the annotation in the main
  - Return a few slides earlier to see this change

Account Service Improvements

- We can now edit the existing methods in the AccountController to handle customers linked to the accounts

- getAllAccounts methods will request for the matching customer and add its details to the customer property of the account

```
@GetMapping("/accounts")
public List<Account> getAllAccounts()
{
    List<Account> accountList = accountRepository.findAll();
    accountList.forEach(account -> {
        account.setCustomer(customerClient.getCustomerById(account.getCustomerId()));
    });

    return accountList;
}
```


www.mbds-fr.org

Account Service Improvements

- Same goes for the getAccountById method

```
@GetMapping("/account/{id}")
public Account getAccountById(@PathVariable String id)
{
    Account accountInstance = accountRepository.findById(id).orElse(null);
    if (accountInstance == null) {
        return null;
    }
    Customer customer = customerClient.getCustomerById(accountInstance.getCustomerId());
    accountInstance.setCustomer(customer);

    return accountInstance;
}
```

Account Service Improvements

- Now the last thing missing is the dynamic account data generation, we need to change a few things

```
@Bean
    CommandLineRunner commandLineRunner(AccountRepository accountRepository, CustomerClient customerClient) {
        return args -> {
            customerClient.getAllCustomers().forEach(customer -> {
                Account accountInstance = Account.builder()
                        .customerId(customer.getId())
                        .id(UUID.randomUUID().toString())
                        .balance(Math.random()*1000)
                        .dateCreated(LocalDate.now())
                        .currencyType(CurrencyType.EUR)
                        .build();
                accountRepository.save(accountInstance);
            });
        };
    }
```

Account Service Improvements

- All done !
- Now when creating accounts, we are using real customer data
- When testing the account endpoints
  - /accounts now returns the matching customer
  - /account/{id} is also doing the same
- That’s it for the internal services links
- Let’s improve the code we wrote to make it more compliant with the best practices

Best Practices

Code refactor

Best Practices

- Since the beginning, the **code we wrote do not respect the best practices**

- **We should never request any data from a Controller (SRP)**, that's something we are going to change with the introduction of a **Service** dedicated to data retrieving from the repositories

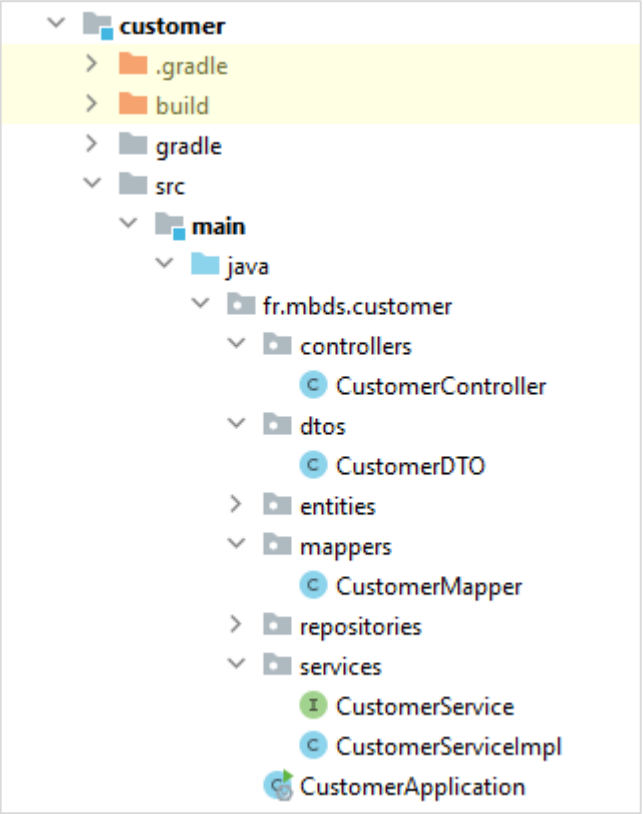
- **We should never use Entities to hand data from the repositories to another component**
  - Instead, we need to create a **DTO** (Data Transfer Object). This object will be responsible of the data forwarding
  - To create these DTOs and be able to convert DTOs back to Entities, we will also need a **Mapper**

Best Practices

- Finally, the Service we will create needs implements Service Interface we will create to expose DAO manipulations methods
  - Aims to respect DIP
  - Provide better segmentation
  - Aims to improve testability
  - Allow to easily swap the Service implementation if needed in the future

Best Practices

- To summarize, we need to create new files
  - CustomerDTO
  - CustomerService (Interface)
  - CustomerServiceImpl
  - CustomerMapper
- Then we'll need to edit the CustomerController to use this newly created Service



Best Practices – CustomerDTO

```
@Data @NoArgsConstructor @AllArgsConstructor
public class CustomerDTO {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
}
```

Best Practices – Mapper

```
@Component
public class CustomerMapper {
    public CustomerDTO toDTO(Customer customer)
    {
        return new CustomerDTO(customer.getId(), customer.getFirstName(),
                               customer.getLastName(), customer.getEmail());
    }

    public Customer toCustomer(CustomerDTO customerDTO)
    {
        if (customerDTO != null)
            return new Customer(null, customerDTO.getFirstName(),
                               customerDTO.getLastName(), customerDTO.getEmail());
        return null;
    }
}
```

Best Practices – Service Interface

```
public interface CustomerService {
    List<CustomerDTO> findAll();
    CustomerDTO findById(Long id);
}
```

Best Practices – Service Implementation (1)

UNIVERSITÉ CÔTE D'AZUR

```
@Service
public class CustomerServiceImpl implements CustomerService {

    private final CustomerRepository customerRepository;
    private final CustomerMapper customerMapper;

    public CustomerServiceImpl(CustomerRepository customerRepository, CustomerMapper customerMapper) {
        this.customerRepository = customerRepository;
        this.customerMapper = customerMapper;
    }

    @Override
    public List<CustomerDTO> findAll() {
        return customerRepository
                .findAll()
                .stream()
                .map(customerMapper::toDTO)
                .collect(Collectors.toList());
    }
```

Best Practices – Service Implementation (2)

```
    …

    @Override
    public CustomerDTO findById(Long id) {
        Customer customer = customerRepository.findById(id).orElse(null);
        if (customer != null)
            return customerMapper.toDTO(customer);
        return null;
    }
}
```


www.mbds-fr.org

Best Practices - CustomerController

UNIVERSITÉ CÔTE D'AZUR

- Should now use the new Service instead of the Repository

```
public class CustomerController {

    CustomerService customerService;

    public CustomerController(CustomerService customerService) {
        this.customerService = customerService;
    }
    …
```

Best Practices

Code refactor bis

Best Practices – Account Service

- Now reproduce the same exact refactoring for the Account Service
- You will end up with 2 nice microservices respecting good practices
- Next, we will introduce edge microservices to make this architecture shine

Assignment

What you have to do ?

Gateway Service

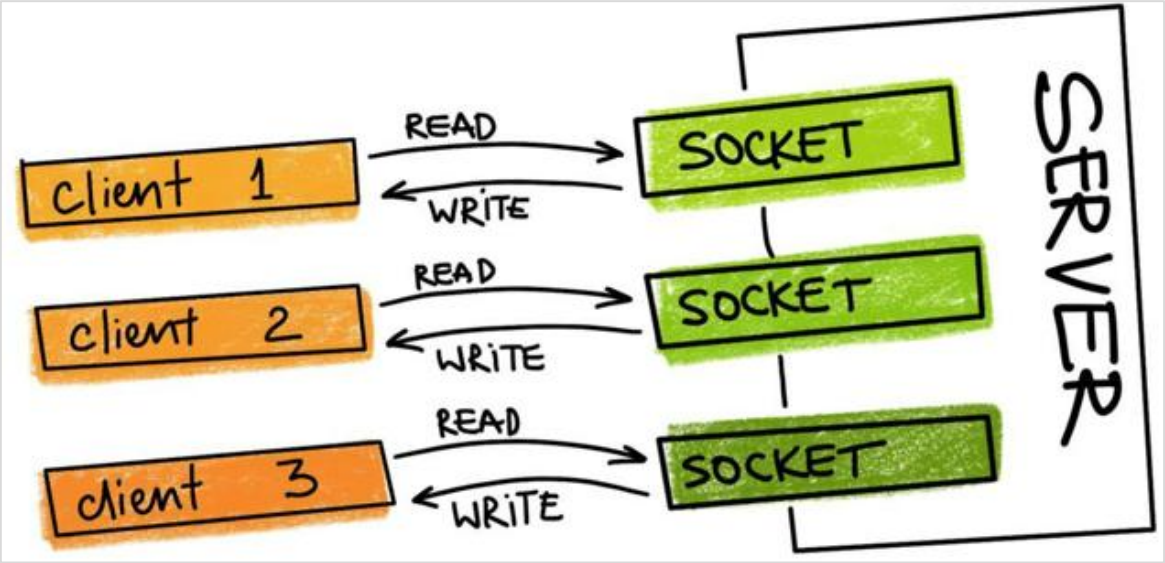
New Module

Gateway Service

- To be able to contact different services without knowing their IP
  - Crucial in microservices ecosystem as a given service can be instantiated multiple times
  - Need to handle the routing without having to know each instance address
- First step is to enable a Gateway service
- Later we will link it to a Discovery Service to make it autonomous
- Many other functionalities can be handled by the Gateway (later)

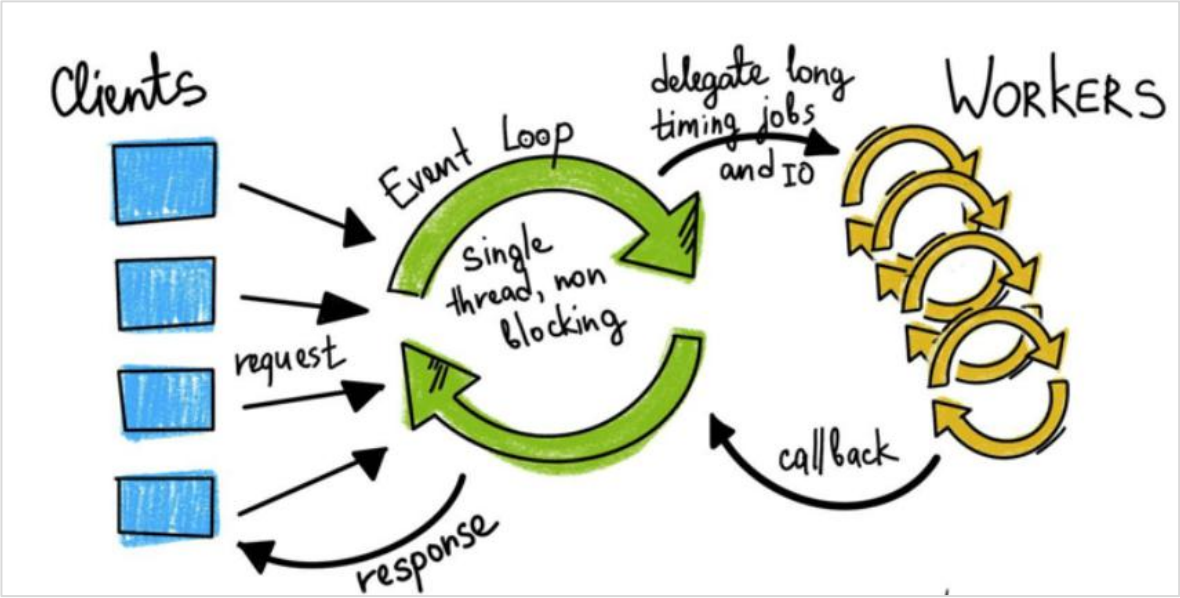
Gateway Service – Gateway Types

- Multi-Thread, Blocking Input/Output
  - Ex : Zuul
  - Not able to scale vertically
  - No warranty all the server resources will be used → Thread limit
  - Can be improved through horizontal scalability



Gateway Service – Gateway Types

- Single-Thread, Non-Blocking Input/Output
  - Spring Cloud Gateway (Spring Reactive)
  - Will accept as much request as the memory can handle
  - Can be improved through vertical scalability



Gateway Service – Spring Cloud Gateway

UNIVERSITÉ CÔTE D'AZUR

- Routing system
- Route defined by
  - URI : destination address
  - Predicate : Conditions a request must satisfy to be forwarded
  - Filters : Used to alter the selected request
- Structure of a HTTP Request
  -

Method	Host	Path	Params
Get	http://site.com	/new	?lang=fr

Gateway Service – Spring Cloud Gateway

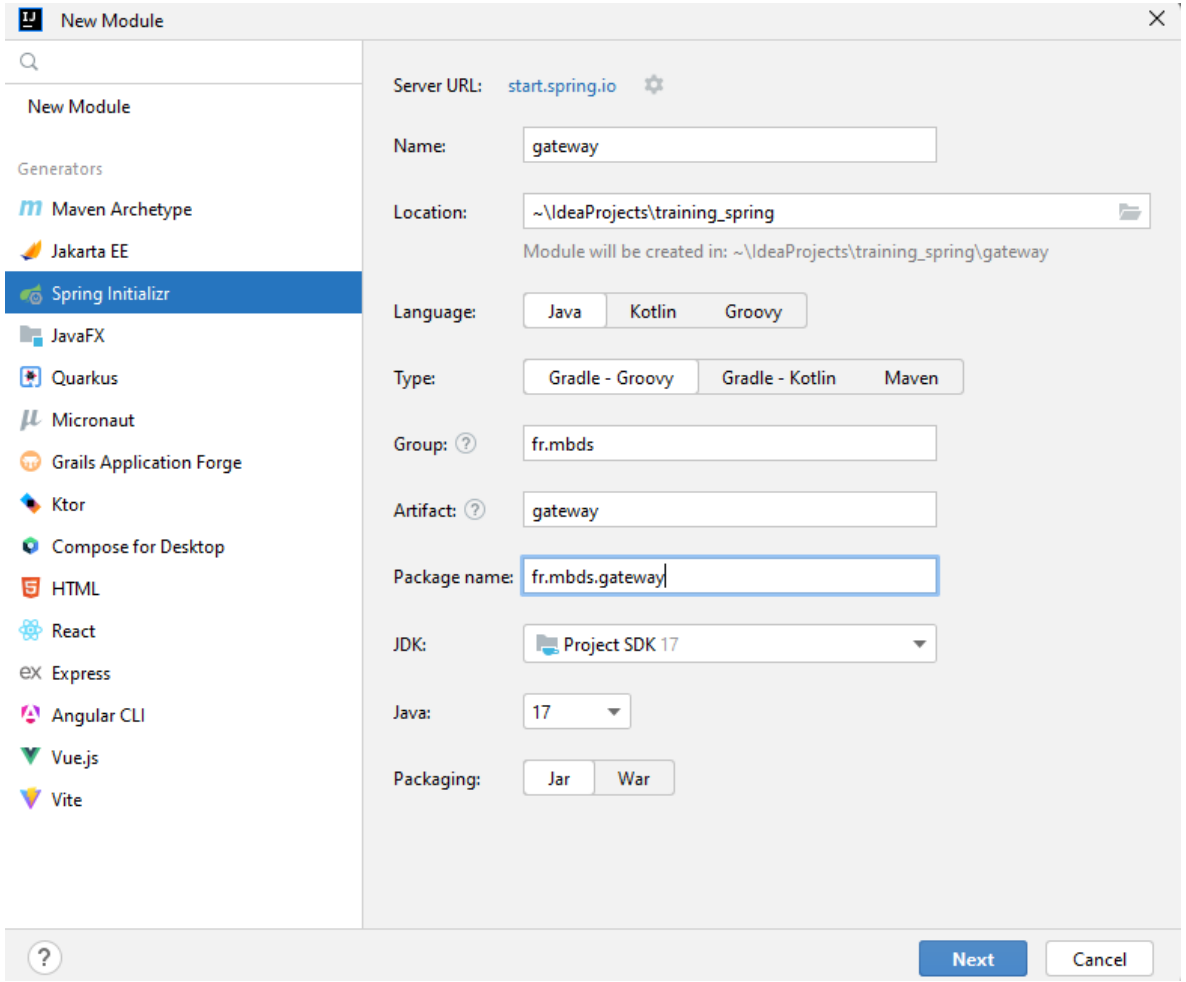
- Predicates can be used to test multiple values
  - Host
  - Path
  - Method
  - Dates : After / Before / Between
  - Metadata : Cookie, Header, Query
  - RemoteAddr
  - …

Gateway Service – Spring Cloud Gateway

- Filters
  - AddRequestHeader
  - AddRequestParameter
  - AddResponseHeader
  - AddResponseParameter
  - CircuitBreaker
  - and many more …

Gateway Service

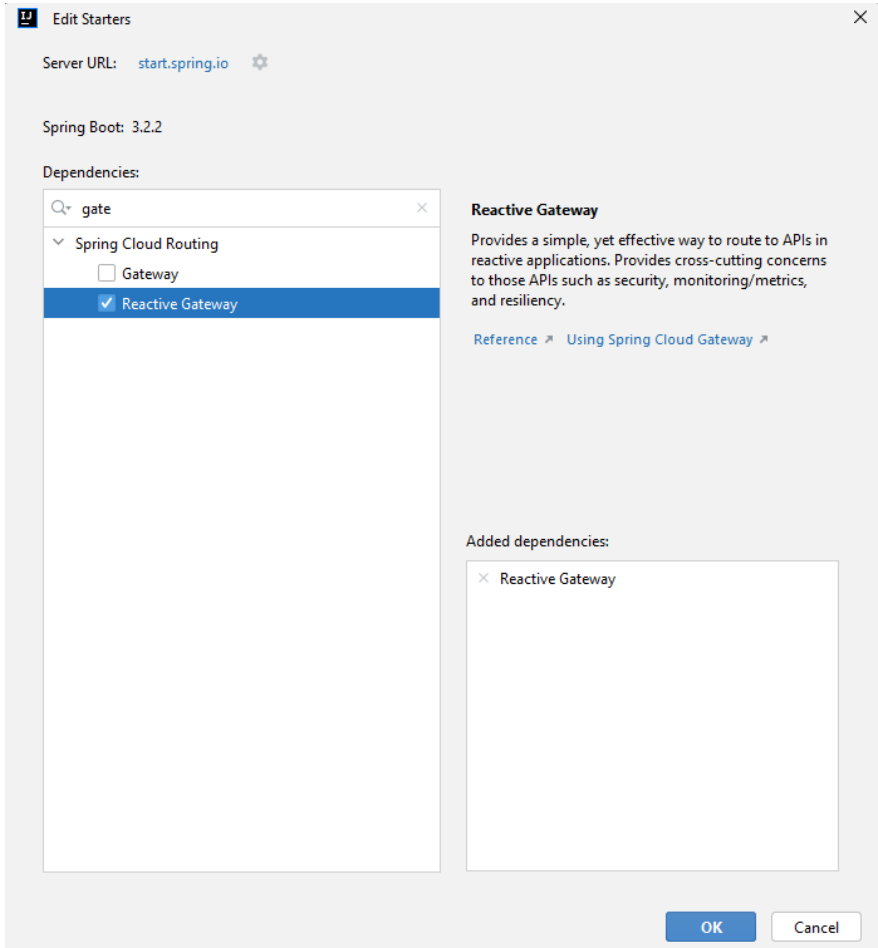
- Start by creating a new module
  - This module will be a classic Spring boot module
  - Add necessary dependencies



Gateway Service

- Take care to use the **Reactive Gateway**
- Based on Non-Blocking Input/Output
- Set basic configuration for now

```
spring.application.name=gateway
server.port=8888
```



Gateway Service

- Now we need to declare some new routes so that the Gateway can forward request to the appropriate service

- Routes is an array you should define in the config, easier with yml

```
spring.cloud.gateway.routes[0].id = customer
spring.cloud.gateway.routes[0].uri = http://localhost:8091/
spring.cloud.gateway.routes[0].predicates[0] = Path=/customers/**,/customer/**
spring.cloud.gateway.routes[1].id = account
spring.cloud.gateway.routes[1].uri = http://localhost:8092/
spring.cloud.gateway.routes[1].predicates[0] = Path=/accounts/**,/account/**
```

Gateway Service

- Its easier by far with yml

```yml
spring:
  cloud:
    gateway:
      routes:
        - id: customers
          uri: http://localhost:8091/
          predicates:
            - Path=/customers/**,/customer/**
        - id: accounts
          uri: http://localhost:8092/
          predicates:
            - Path=/accounts/**,/account/**
```

Gateway Service

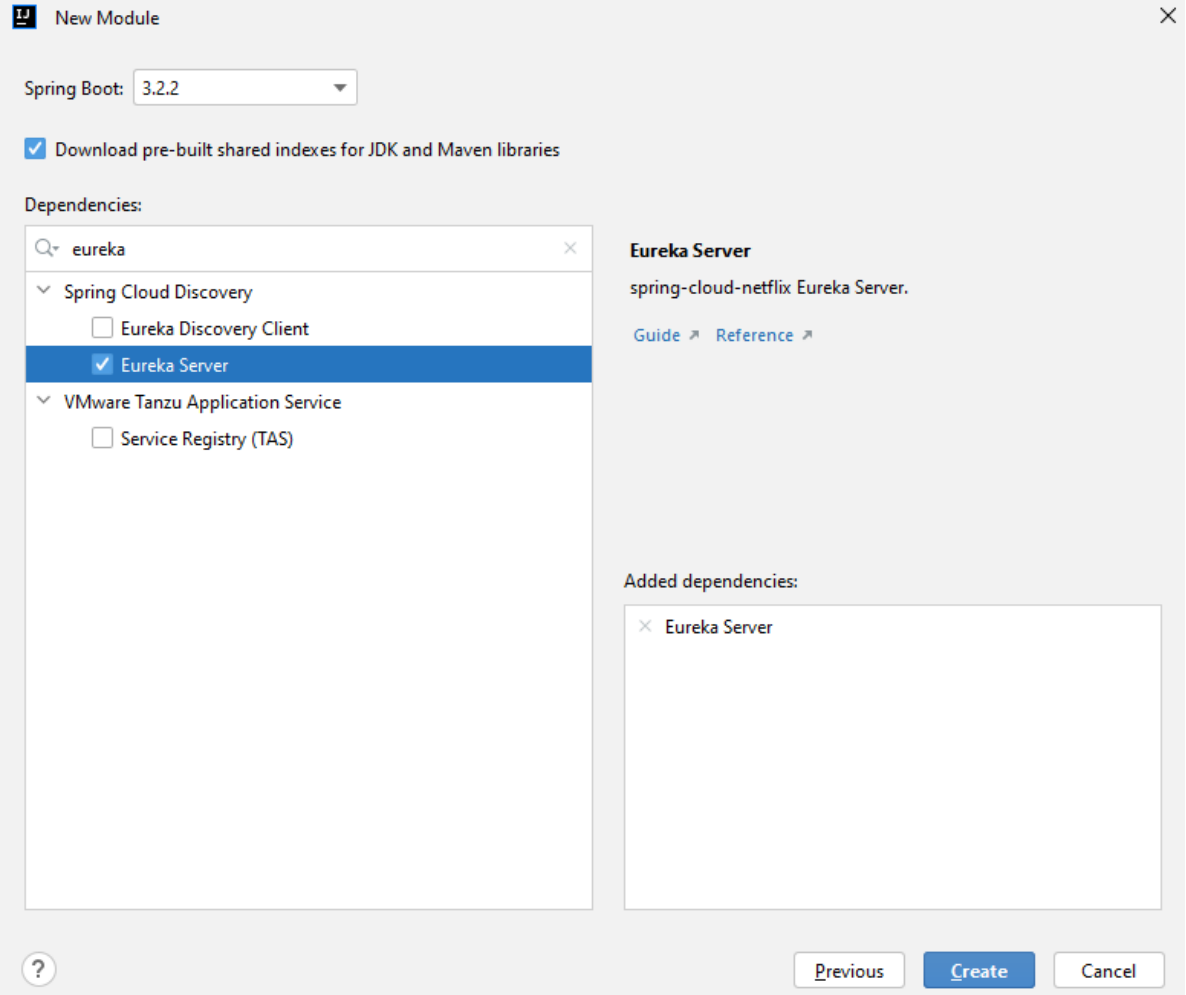
- Once done, you can try and access services from the gateway
- Address will be the base address of the gateway : http://localhost:8888
- Followed by the predicates you have set in the configuration
  - /customers/** and /customer/** will lead you to the Customer Service
  - /accounts/** and /account/** will lead you to the Account Service
- Now try for yourself, and **save this step as the branch 5-1-static-routing**

Discovery Service

New Module

Discovery Service

- Now, we want to make the routing more dynamic
- Let's add a new module, this module will be our Discovery Service
- The only dependency we need is the “Eureka Server”



Discovery Service

- Add a new annotation to the main of this Service
- Declare this service as a Eureka Server

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryApplication {

    public static void main(String[] args) {
        SpringApplication.run(DiscoveryApplication.class, args);
    }
}
```


www.mbds-fr.org

Discovery Service

- Set the configuration as follow

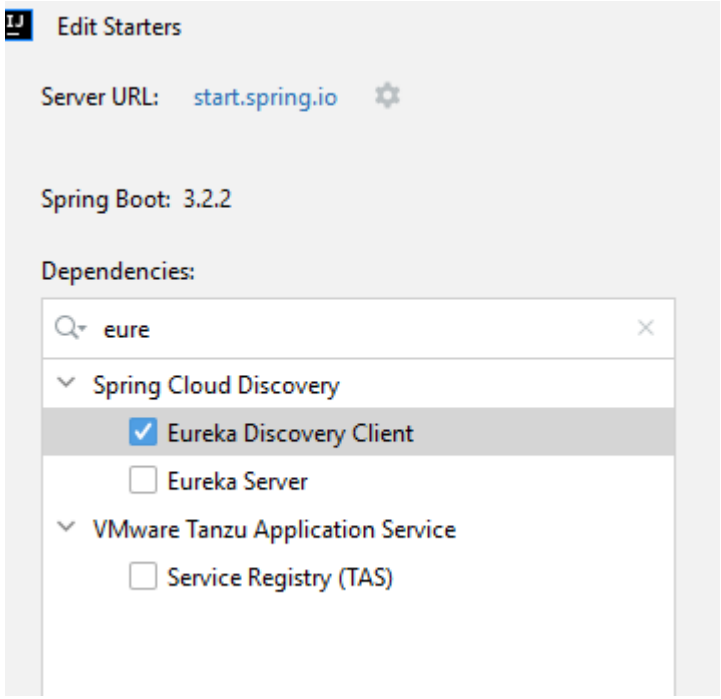
```
# Prevent Eureka Server to register itself as a Client
eureka.client.fetch-registry = false
# Prevent Eureka Server to register itself in the service registry
eureka.client.register-with-eureka = false
# Note: we would enable both if we had many Eureka Servers

server.port = 8761
```


www.mbds-fr.org

Discovery Service

- Our Discovery Service is ready, now add a dependency to all the other services
- Each service should be edited to add Eureka Discovery Client



Discovery Service

- Once done you should be able to see all the available services

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ACCOUNT	n/a (1)	(1)	UP (1) - ggalli:account:8092
CUSTOMER	n/a (1)	(1)	UP (1) - ggalli:customer:8091
GATEWAY	n/a (1)	(1)	UP (1) - ggalli:gateway:8888

Discovery Service

- Note that Services as linked with the machine name instead of the IP address
- To change this, just add in each service config : `eureka.instance.prefer-ip-address=true`
- Link will stay the same, but the link will forward to an IP

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ACCOUNT	n/a (1)	(1)	UP (1) - ggalli:account:8092
CUSTOMER	n/a (1)	(1)	UP (1) - ggalli:customer:8091
GATEWAY	n/a (1)	(1)	UP (1) - ggalli:gateway:8888

192.168.22.127:8092/actuator/info

Discovery Service

- That's it for the Discovery Service … for now …
- **Save this step as the branch 5-2-discovery-service**

Dynamic Gateway

Enhance the Gateway Service

Dynamic Gateway

- Now the Discovery Service is set, we can use it from the Gateway Service
  - Will get rid of the static routes we wrote
  - Will update live with the Discovery Service
  - Will be able to handle more than one instance of a service simultaneously

Dynamic Gateway

- Remove the Routes declaration in the Gateway config file
- To enable dynamic routing the Discovery Service you can use either solution
- Add a new line in the configuration as follow

```
spring.cloud.gateway.discovery.locator.enabled=true
```

- Or set this dynamic behavior by adding this to the main of the Gateway Service

```
@Bean
DiscoveryClientRouteDefinitionLocator discoveryRoutes(ReactiveDiscoveryClient rdc, DiscoveryLocatorProperties dlp) {
    return new DiscoveryClientRouteDefinitionLocator(rdc, dlp);
}
```

Dynamic Gateway

- That's it, your gateway can reach any service registered with the discovery service
- Try it
  - To reach a service you will have to use its name in uppercase

    ex : http://localhost:8888/CUSTOMER/customers

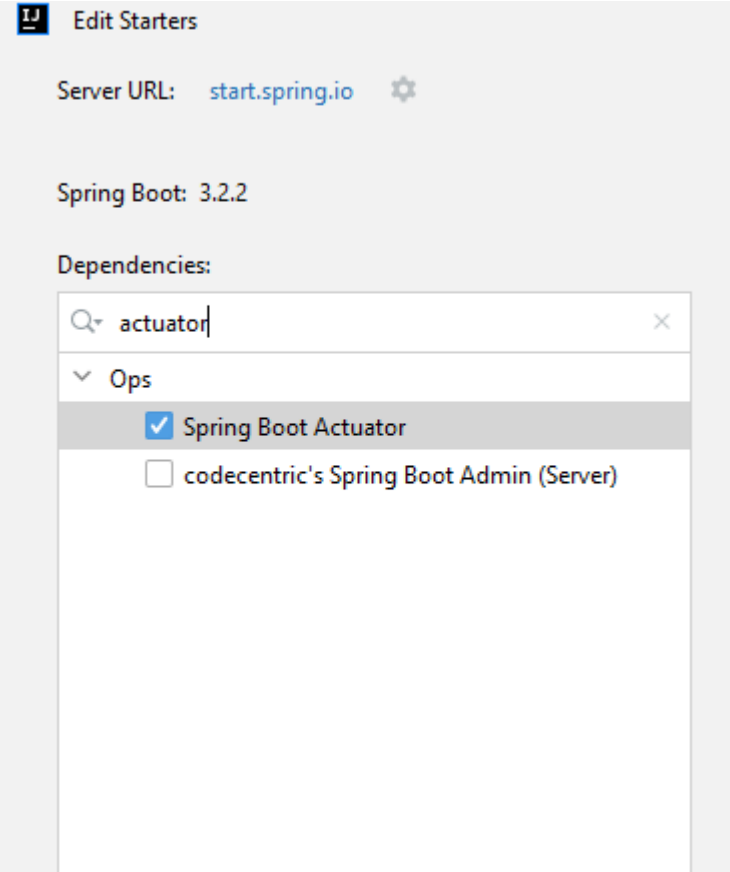
    or : http://localhost:8888/ACCOUNT/accounts
- Once done **save this step as the branch 5-3-dynamic-gateway**

Actuator

Monitoring

Actuator

- Let you monitor and interact with your services
- Huge set of built-in endpoints
- Add actuator to your services to enable the monitoring
- Few endpoints enabled by default



Actuator

- Edit the config to enable / disable endpoints
- To enable all endpoints

```
management.endpoints.web.exposure.include=*
```

- Enable all endpoints at least on one service and check the result
- Display the list visiting this address : http://localhost:8888/customer/actuator
- Most important one is /health, to keep track of the service state
- Once done **save this step as the branch 6-actuator**

www.mbds-fr.org

Circuit Breaker

Resilience4J

Circuit Breaker

- Issue : When we created the link between Account and Customer, we made Account Service dependent on the Customer Service state

- If the Customer Service is down, we are not able to use the Account Service endpoints
  - Try and stop the Customer Service
  - Try to display the accounts
  - Error 500 is shown

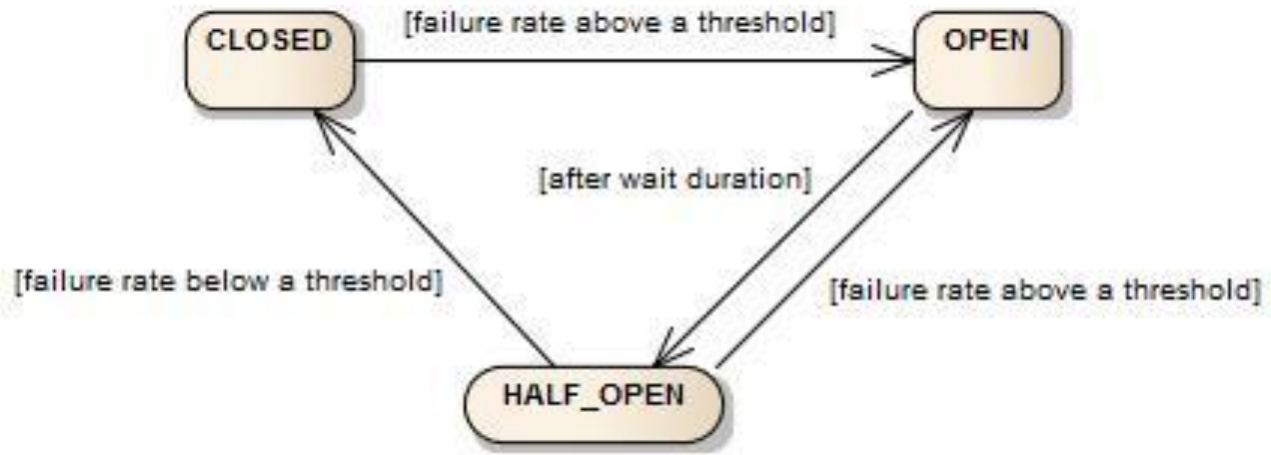
- Look at the Account Service logs, and you will see the error origin

Circuit Breaker

- We need to implement something that will make Account Service more resilient
- Circuit breakers are made for this
- The one we are going to use : Resilience4J
- Can be use in different ways
  - Circuit breaker
  - Rate limiter
  - Time limiter
  - Retry mechanism … and much more

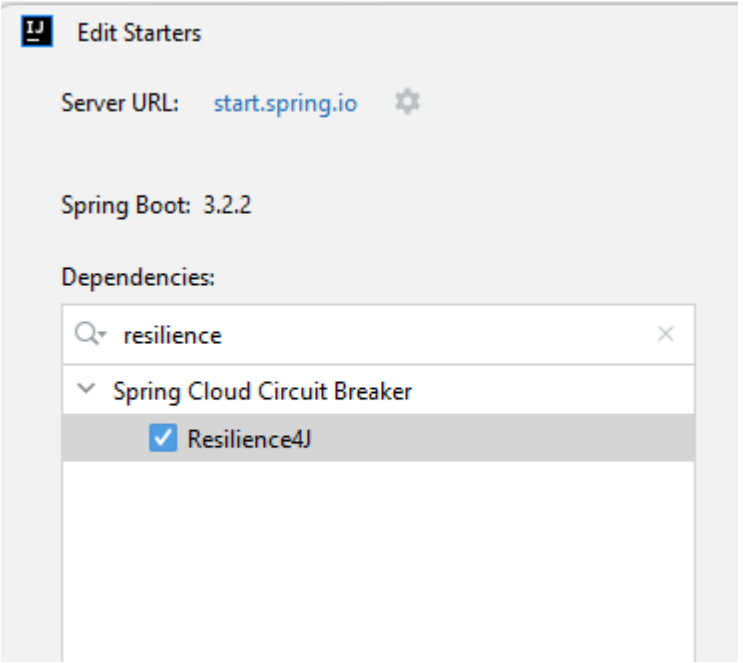
Circuit Breaker

- It’s a design pattern working with 3 states
  - Open
  - Closed
  - Half open



Circuit Breaker

- Add new dependency for Resilience4J
- Of course, concerning only the Account Service
- Use it to protect the Feign Client methods
  - getCustomerById(Long id)
  - getAllCustomers()



Circuit Breaker

UNIVERSITÉ CÔTE D'AZUR

- Simply add a new annotation to enable the circuit breaker

```
@FeignClient(name = "CUSTOMER")
public interface CustomerClient {

    @GetMapping("/customer/{id}")
    @CircuitBreaker(name = "customer", fallbackMethod = "getDefaultCustomer")
    public Customer getCustomerById(@PathVariable Long id);

    @GetMapping("/customers")
    @CircuitBreaker(name = "customer", fallbackMethod = "getAllCustomers")
    public List<Customer> getAllCustomers();
```

Circuit Breaker

- Then provide an implementation for these default methods

```
default Customer getDefaultCustomer(Long id, Exception exception)
{
    Customer customer = new Customer();
    customer.setId(id);
    customer.setFirstName("Not available");
    customer.setLastName("Not available");
    customer.setEmail("Not available");

    return customer;
}

default List<Customer> getAllCustomers(Exception exception)
{
    return List.of();
}
```


www.mbds-fr.org

Circuit Breaker

- With this change, if Customer Service is stopped, Account Service can respond

localhost:8888/ACCOUNT/accounts


```
[
  - {
      id: "7812586c-b03b-4a43-a2f4-84dd9c1e6595",
      balance: 99.05350103589971,
      currencyType: "EUR",
      dateCreated: "2024-02-15",
      customerId: 1,
    - customer: {
        id: 1,
        firstName: "Not available",
        lastName: "Not available",
        email: "Not available"
      }
    },
```


- Once done **save this step as the branch 7-circuit-breaker**

Config Service

New Module

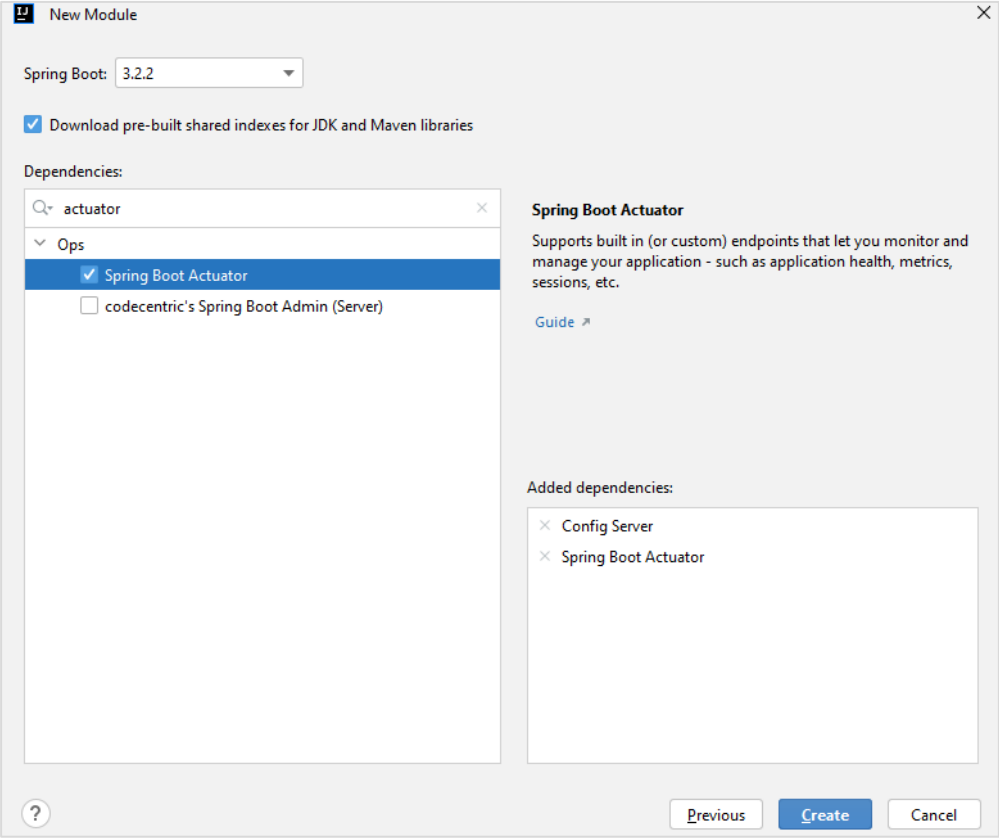
Config Server

UNIVERSITÉ CÔTE D'AZUR

- Configuration is a tedious and time-consuming task
- To alleviate this, we are going to build a Service that will be dedicated to configuration aspects
- This Config Microservice will have the responsibility to provide every service with their optional configuration
- Also, to make it easier to use, we're going to use Git to store configuration files so that you can modify them without having to get your hands inside the project.

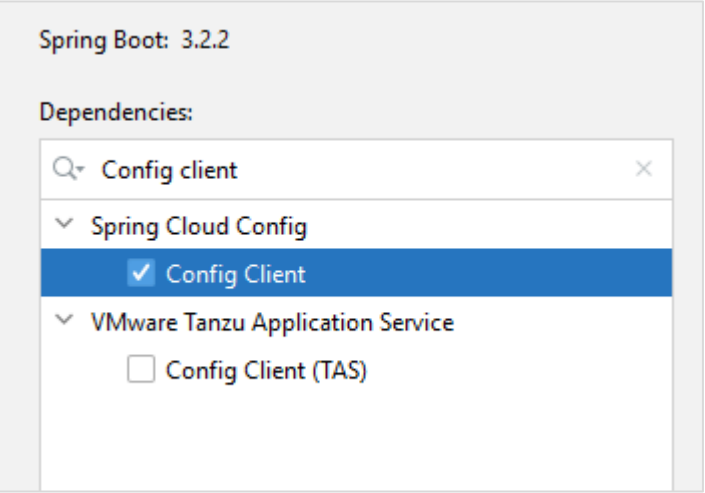
Config Server

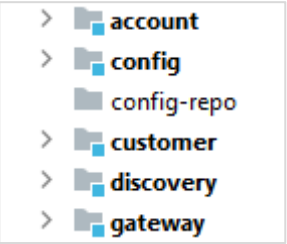
- Start creating a Config Service with two dependencies : Config Server & Actuator



Config Server

- Then add the Config Client dependency to all services you want to
- Next, we will create a simple directory in the main “Empty” project
  - Let’s call it “config-repo”
- This is how your project should look like





www.mbds-fr.org

Config Server

- Add an annotation to the Config Service main

```
@SpringBootApplication
@EnableConfigServer
public class ConfigApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigApplication.class, args);
    }

}
```


www.mbds-fr.org

Config Server

- Then set the Config Service configuration

```
spring.application.name=config
server.port=9999

# Use this to enable fetching on git
#spring.cloud.config.server.git.uri=file:/C:/Users/gregg/IdeaProjects/training_spring/config-repo/

# If you want to use a local directory use this instead
spring.cloud.config.server.native.searchLocations=file:/C:/Users/gregg/IdeaProjects/spring/config-repo/
spring.profiles.active=native
```

Config Repo

- In the config-repo directory we created, we can add a lot of things
- A general config file for all the services : application.properties
- A configuration file for a specific service
  - customer.properties
  - account.properties
  - gateway.properties
  - discovery.properties
- We can also differentiate these configuration file depending on the environment
  - customer-dev.properties
  - customer-test.properties
  - customer-prod.properties

www.mbds-fr.org

Config Repo

- Let's create a few files and put a few values just for testing purposes
- Then, start the service and try to retrieve these config files
- http://localhost:9999/application/default
  - Will fetch the default config from application.properties
- http://localhost:9999/customer/default
  - Will fetch the default config from customer.properties
- http://localhost:9999/account/default
  - Will fetch the default config from account.properties

www.mbds-fr.org

Config Client

- To be able to retrieve the configuration from other services you will need
  - The dependency Config Client you already added
  - A configuration line to specify the Config Server address

```
spring.config.import=optional:configserver:http://localhost:9999
```

- We are all set, it’s not time to move all the config we can to this config-repo
- Then test everything to make sure it’s rock solid

www.mbds-fr.org

Config Client – Hot reload config

- Finally, last issue
- If we are updating the config in the config-repo directory, configuration changes won't be applied to the services
- We need to implement some sort of refresh mechanism
- Test this issue with these steps
  - Start config & customer
  - Change customer configuration in the config-repo
  - Check if the customer service is showing the configuration changes

Config Client – Hot reload config

- To check a configuration value, many solutions exists
- One of them is the following
  - Create a REST Controller dedicated for this test
  - Declare class attributed for the configuration values
  - Bind them using the @Value annotation
  - Finally create a method to display those values

Config Client – Hot reload config

UNIVERSITÉ CÔTE D'AZUR

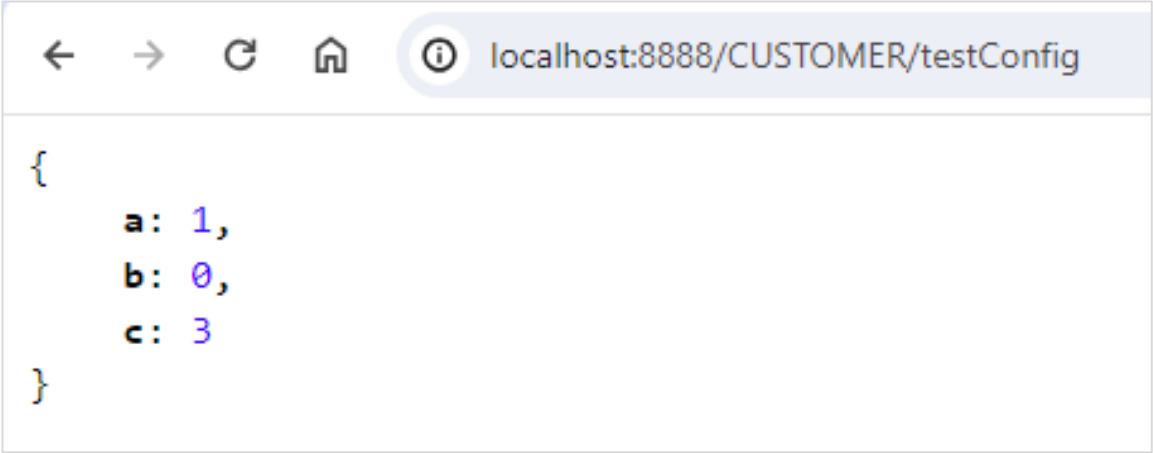
- It will look something like this

```
@RestController
@RefreshScope
public class ConfigTestController {
    @Value("${a}")
    private int a;
    @Value("${b}")
    private int b;
    @Value("${c}")
    private int c;

    @GetMapping("/testConfig")
    public Map<String, Integer> testConfig()
    {
        return Map.of("a",a,"b",b,"c",c);
    }
}
```

Config Client – Hot reload config

- Now check the url set in the controller to see if the configuration is right



- Here you can see everything is right
  - "a" and "c" values were read in the application.properties file
  - "b" value was overridden by the customer.properties values

Config Client – Hot reload config

- Finally, if we want the configuration to “hot reload” we need to add a few things
  - We will need to call actuator on the service and ask for a refresh
    - Request is a POST request
    - Target url is /actuator/refresh
  - Once this is done, one thing is still missing, we need to inform our controller that the bean must be refreshed, and to do this, we will need to add an annotation
    - Simply add @RefreshScope annotation to the controller in charge of the config display
- Restart everything and test it
- Once done **save this step as the branch 8-config-server**

Already done ?!

Bonus service

Bonus Service

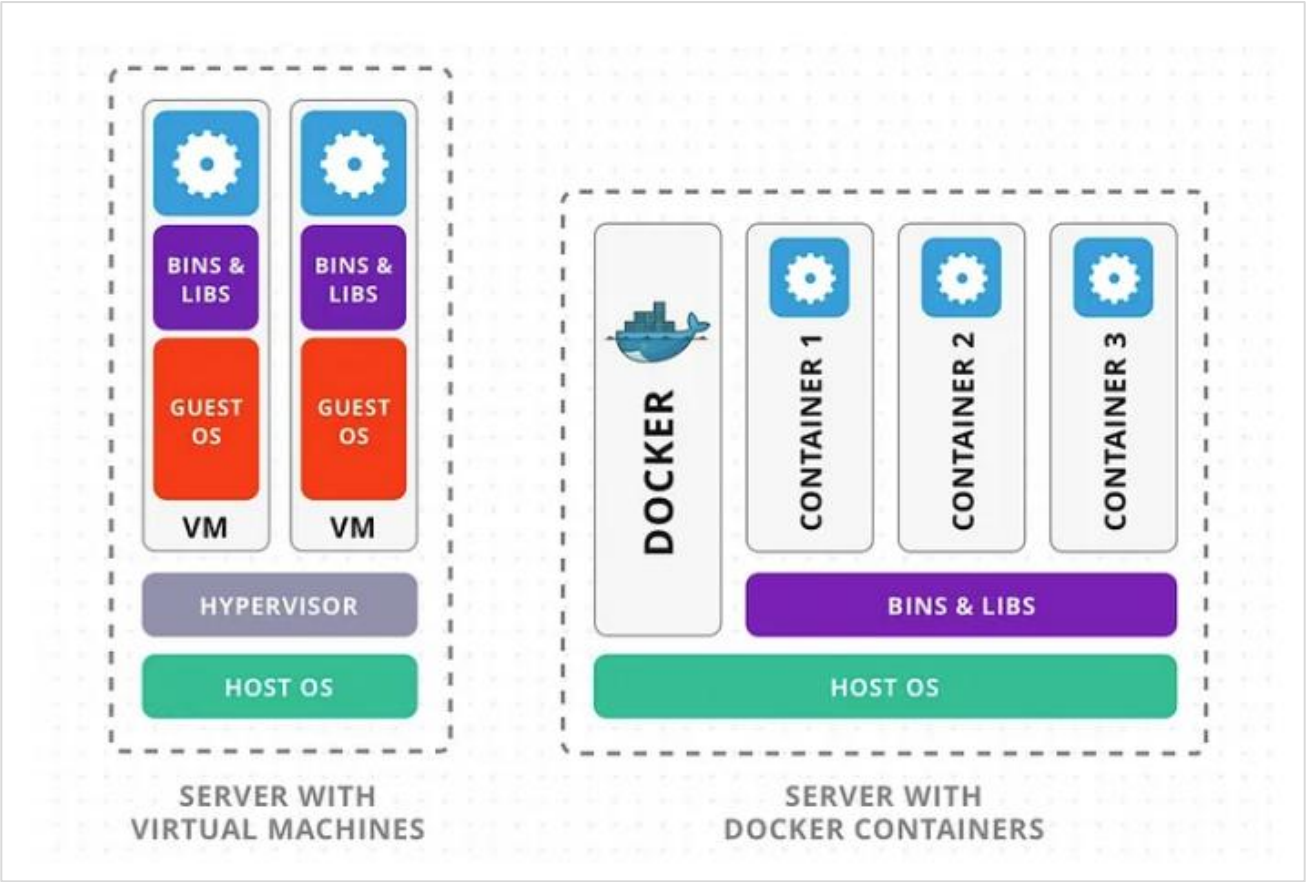
- If you are done, this is a bonus :)
- Add a new Service to handle wire transfer
  - This new microservice will declare some attributes like
    - A source account
    - A destination account
    - An amount
    - A date
  - This service will need to “contact” the account service (Feign) and should respect every step we worked
  - Finally, add a POST endpoint that will handle the wire transfer creation
  - Once done **save this step as the branch 9-bonus**

Docker

Dockerize the project

Docker

- Platform used to start applications within containers



Docker vs VM

- Docker
  - Boots in a few seconds
  - No virtualization, less memory
  - Easy to deploy
  - Nothing to change no matter the environment
- VM
  - Up to a few minutes to boot
  - Need to boot an OS
  - Not subject to interferences
  - Pure native performances

Docker for our project

- Build images containing the different services we created
- Make sure the images will be able to communicate when isolated in container
- Use a mechanism to orchestrate everything
- Will need a few tweaks and some config changes

Docker - Install

- First step is to install Docker / Docker desktop : https://www.docker.com/
- Once done, start the program
- Now back to the code

Docker – Commands & Glossary

- But before, here is a few commands we will use as well as their options
- Dockerfile : is a file describing how an image should be built
- build : will build the image based on the Dockerfile you created
  - -t : set the name of the image
  - Example : `docker build –t discovery:v1`
- run : will run the built image, you need to specify the image name
  - -p : specify the port forwarding policy
  - -d : runs the image as a background process
  - Example : `docker run –d –p 8761:8761 discovery:v1`

Docker – Commands & Glossary

- stop : will stop a container based on its name
  - Example : `docker stop discovery:v1`

- logs : display mentioned container logs
  - Example : `docker logs discovery`

- docker-compose.yml : file in charge of describing how containers should be executed, the starting order, environment variables and many more

Docker – Commands & Glossary

- compose : docker command to interact with the docker-compose.yml
  - docker compose up : will run the container on the images described in the docker-compose.yml
    - -d : starts the containers in a background process
    - --build : ask the different images to be rebuilt before the deployment
    - Example : `docker compose up –d –build`
  - docker compose down : will stop the containers started from the docker-compose.yml file
    - Example : `docker compose down`

Docker – Dockerize your project

UNIVERSITÉ CÔTE D'AZUR

- To Dockerize our project we will need to
  - Create a Dockerfile for each service
  - Make sure the config is set correctly
  - Make sure no localhost communication is used
  - Run every service independently
  - Check the logs to make sure everything is running fine
  - Create a docker-compose.yml file to handle all the startup tasks for us
  - Declare some environment variables to handle the config parts we need
  - Enjoy

Docker – Dockerize your project

- Once done and check we will be able to deploy our project anywhere thanks to docker

- If you want to run your project on a new system
  - Checkout the code
  - Run docker compose up
  - That’s it !

Docker – Dockerize your project

UNIVERSITÉ CÔTE D'AZUR

- Here are a few details about the things we will need to do, starting with the Dockerfile

- Keep in mind, if you want this to work, you will have to build your services first with the following command : ./gradlew build

```
FROM openjdk:17-oracle
VOLUME /tmp
ADD build/libs/*.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Docker – Dockerize your project

- And here is an example of a docker-compose.yml file
- This is just a little part of the config we will need, the other stuff will be shown live

```yaml
services:
  discovery:
    build: ./discovery
    container_name: discovery
    ports:
      - "8761:8761"
    expose:
      - "8761"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8761/actuator/health"]
      interval: 5s
```

Docker – Dockerize your project

- One last thing, to be able to run your project locally and with docker, you will have to use environment variable, those variables can be set as follow

- We will use this system for the discovery service url as well as the config service url

```
eureka.client.service-url.defaultZone=${DISCOVERY_SERVICE_URL:http://localhost:8761/eureka}
```

```
spring.config.import=optional:configserver:${CONFIG_SERVICE_URL:http://localhost:9999}
```

- Once done **save this step as the branch 10-docker**

All good !

Resource

- https://luminousmen.com/post/asynchronous-programming-blocking-and-non-blocking
- https://hanwenzhang123.medium.com/docker-vs-virtual-machine-vs-kubernetes-overview-389db7de7618

Full docker-compose.yml file

```yaml
services:
  discovery:
    build: ./discovery
    container_name: discovery
    ports:
      - "8761:8761"
    expose:
      - "8761"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8761/actuator/health"]
      interval: 5s
…
```

Full docker-compose.yml file

```yaml
  config:
    build: ./config
    container_name: config
    ports:
      - "9999:9999"
    expose:
      - "9999"
    depends_on:
      discovery:
        condition: service_healthy
    environment:
      - DISCOVERY_SERVICE_URL=http://discovery:8761/eureka
    healthcheck:
      test: [ "CMD", "curl", "-f", "http://localhost:9999/actuator/health" ]
      interval: 5s
```

Full docker-compose.yml file

```yaml
customer:
  build: ./customer
  container_name: customer
  ports:
    - "8091:8091"
  expose:
    - "8091"
  depends_on:
    config:
      condition: service_healthy
  environment:
    - DISCOVERY_SERVICE_URL=http://discovery:8761/eureka
    - CONFIG_SERVICE_URL=http://config:9999
  healthcheck:
    test: [ "CMD", "curl", "-f", "http://localhost:8091/actuator/health" ]
    interval: 5s
```

Full docker-compose.yml file

```yaml
account:
  build: ./account
  container_name: account
  ports:
    - "8092:8092"
  expose:
    - "8092"
  depends_on:
    customer:
      condition: service_healthy
  environment:
    - DISCOVERY_SERVICE_URL=http://discovery:8761/eureka
    - CONFIG_SERVICE_URL=http://config:9999
  healthcheck:
    test: [ "CMD", "curl", "-f", "http://localhost:8092/actuator/health" ]
    interval: 5s
```


www.mbds-fr.org

Full docker-compose.yml file

```yaml
gateway:
  build: ./gateway
  container_name: gateway
  ports:
    - "8888:8888"
  expose:
    - "8888"
  depends_on:
    account:
      condition: service_healthy
  environment:
    - DISCOVERY_SERVICE_URL=http://discovery:8761/eureka
    - CONFIG_SERVICE_URL=http://config:9999
```


www.mbds-fr.org